

Automated Acronym Understanding and CCC-Based Concept Mapping in Scientific Text

Complete Project Report — Phases 1 through 6

August 2026

Contents

Phase 1 — Problem Understanding	3
1. Problem Statement	3
2. Motivation	3
3. Objectives	4
4. Expected Output	4
5. Scope	5
6. Expected Contributions	5
Phase 2 — Literature Review	7
Step 2.1 — Collected Papers (Base Set)	7
Step 2.2 — Paper Extraction Table	7
Step 2.3 — Categorization of Papers	10
Step 2.4 — Evolution of the Research (Chronological)	10
Step 2.5 — Comparison Table	11
Step 2.6 — Common Limitations Across Papers	13
Step 2.7 — Literature Review (Narrative Summary / Related Work Draft) . .	13
Initial Research Gap (carried into Phase 3)	14
Deliverables Checklist	15
Phase 3 — Research Gap Analysis	16
Step 3.1-3.2 — Paper-wise Limitations (“What still remains unsolved?”) . .	16
Step 3.3 — Common Missing Features (Capability Checklist)	17
Step 3.4 — Research Gap Statement	18
Step 3.5 — Novelty Statement	18
Step 3.6 — Research Questions (RQs)	19
Step 3.7 — Research Objectives	20
Step 3.8 — Proposed Framework (High-Level Workflow)	20
Deliverables Checklist	22
Phase 4 — Proposed Methodology	23
4.1 — Input Specification	23
4.2 — Output Specification	23

4.3 — System Modules	24
4.4 — Data Flow	24
4.5 — Module Design	25
4.6 — Complete System Architecture	27
4.7 — Technology Stack	28
4.8 — Evaluation Plan	29
Deliverables Checklist	30
Phase 5 — Dataset Collection & Preparation	31
Step 5.1 — Data Sources	31
Step 5.2 — Research Domain	31
Step 5.3 — PDF Collection & Organization	32
Step 5.4 — Metadata	32
Steps 5.5-5.9 — Text Extraction, Cleaning, Sectioning, Sentence Segmentation, Tokenization	33
Step 5.10 — Gold Standard Dataset	33
Step 5.11 — Abbreviation Dictionary	33
Step 5.12 — Concept Dictionary	33
Step 5.13 — Relationship Dataset	33
Step 5.14 — Dataset Validation	34
Step 5.15 — Final Dataset Structure (as created)	34
Deliverables Checklist	35
Phase 6 — Algorithm & Model Design	36
Step 6.1 — Overall Processing Pipeline	36
Step 6.2 — Algorithm: PDF Processing	37
Step 6.3 — Algorithm: Text Preprocessing	37
Step 6.4 — Algorithm: Abbreviation Identification (Module 4)	37
Step 6.5 — Algorithm: Long Form Extraction (Module 5)	38
Step 6.6 — Algorithm: Acronym Disambiguation (Module 6)	38
Step 6.7 — Algorithm: Concept Extraction (Module 7) — <i>novel</i>	39
Step 6.8 — CCC Mapping Algorithm (Module 8) — <i>core contribution</i>	40
Step 6.9 — Hierarchy Generation Algorithm (Module 9)	41
Step 6.10 — Relationship Extraction Algorithm (Module 8b, feeding Module 10)	41
Step 6.11 — Knowledge Graph Construction (Module 10a)	42
Step 6.12 — Visualization Algorithm (Module 10b)	42
Step 6.13 — Complexity Analysis	42
Step 6.14 — Final Algorithm Flow	44
Model Selection Summary	44
Deliverables Checklist	45

Phase 1 — Problem Understanding

Research Area: Acronym Identification (AI) and Acronym Disambiguation (AD) in Scientific Text

1. Problem Statement

Scientific and technical documents make heavy use of acronyms and abbreviations, but automatically recognizing these short forms and correctly resolving what they mean is still an unsolved problem for NLP systems. Two coupled sub-problems exist:

- **Acronym Identification (AI):** given a sentence, detect which spans are acronyms (short forms) and which spans are their corresponding long forms (definitions).
- **Acronym Disambiguation (AD):** given an acronym that has multiple possible expansions (e.g., “CNN” → *Convolutional Neural Network* or *Cable News Network*), determine the correct long form based on the surrounding context.

Historically, research on these tasks was held back by datasets that were too small, generated through noisy rule-based heuristics, or narrowly confined to the medical/biomedical domain. This made it difficult to train and fairly evaluate deep learning models capable of handling the scale, ambiguity, and domain diversity of real scientific writing (arXiv papers, general text, financial text, etc.). Even after large benchmark datasets (SciAI, SciAD) were introduced, subsequent work revealed additional problems — annotation noise, duplicate examples with conflicting labels, and a persistent performance gap between the best models (~81–94% Macro F1) and human-level accuracy (~96% F1).

Core problem: existing models and resources for acronym identification and disambiguation in scientific text are limited by data quality/scale issues and still fall short of human performance, making reliable automated acronym understanding an open challenge.

2. Motivation

- Acronyms are extremely pervasive in technical writing — roughly 15% of PubMed queries and a large share of scientific/clinical text contain abbreviations.
- Misinterpreting an acronym can silently propagate errors into downstream NLP applications such as question answering, information retrieval, document/glossary generation, and definition extraction.
- Acronyms are frequently ambiguous (the same short form maps to different long forms across domains or even within the same domain), and their correct meaning often depends on distant contextual or syntactic cues rather than the immediately adjacent words.

- Prior benchmark datasets were flawed (small, noisy, single-domain, or containing duplicated/conflicting labels), which both limited model training and biased reported evaluation results.
 - No unified, publicly available, multi-domain tool existed that could both detect and resolve acronyms for a general reader or downstream system — a real practical gap for reading assistance and text-understanding applications.
 - Closing the remaining gap to human-level performance ($\sim 96\%$ F1) is still an active, well-defined, and measurable research target.
-

3. Objectives

1. To build/use large-scale, high-quality, human-annotated datasets for acronym identification and disambiguation in the scientific domain (e.g., SciAI, SciAD, and their refined variants such as SciAD-dedupe).
 2. To design or apply deep learning models that effectively identify acronym-long form pairs in text (e.g., sequence labeling with LSTM-CRF, transformer-based taggers such as XLNet/BERT variants).
 3. To design or apply models that correctly disambiguate acronyms with multiple candidate meanings by leveraging sentence-level context, syntactic structure (dependency trees), and/or pre-trained language model representations (general-domain and domain-specific).
 4. To evaluate and benchmark these models using standard metrics (Precision, Recall, Macro-F1) against existing baselines and human performance.
 5. To identify and, where possible, correct data quality issues (e.g., duplication, conflicting labels) that bias evaluation.
 6. To explore techniques — such as adversarial training, ensembling, task-adaptive pretraining, and dual-path (general + domain-specific) architectures — that improve robustness and push performance closer to human-level accuracy.
 7. (Optionally) To package the resulting approach into a usable, multi-domain, deployable system for real-world acronym lookup/disambiguation.
-

4. Expected Output

- A working acronym identification model that, given raw scientific text, outputs the spans of short forms and their long forms.
- A working acronym disambiguation model that, given an ambiguous acronym and its sentence context, predicts the correct long-form meaning from a set of candidates.
- Quantitative evaluation results (Precision, Recall, Macro-F1) benchmarked against established baselines (e.g., LSTM-CRF, GAD, BERT/SciBERT/RoBERTa variants, hdBERT, XLNet ensembles) and against human-level performance.
- A comparative analysis highlighting which techniques (syntactic/dependency information, adversarial training, ensembling, domain-specific vs. domain-agnostic pretraining, retrieval-based disambiguation) contribute most to performance gains.

- Documentation of any data quality issues found (e.g., duplicate or conflicting examples) and how they were handled.
 - (Optionally) A demonstrable end-to-end pipeline or interface for acronym detection and disambiguation.
-

5. Scope

In scope: - Acronym identification and disambiguation for scientific/academic text (primarily English, arXiv-style papers), following the SciAI/SciAD benchmark framing established in the surveyed literature. - Use of established datasets (SciAI, SciAD, SciAD-dedupe, and related auxiliary sets like AuxAI/AuxAD) and/or construction of similar resources. - Application and comparison of deep learning approaches: sequence labeling (LSTM-CRF), graph-based context modeling (BiLSTM+GCN), transformer fine-tuning (BERT, SciBERT, RoBERTa, ALBERT, ELECTRA, XLNet), adversarial training (FGM), model ensembling, dual-path architectures (hdBERT), and retrieval/embedding-based disambiguation (Siamese networks). - Standard intrinsic evaluation using Precision, Recall, and Macro-F1 against benchmark test sets and reported human performance.

Out of scope: - Full multi-lingual acronym handling (the surveyed work is English-centric). - Deep exploration of non-scientific domains (financial, general Wikipedia, Reddit, biomedical) beyond what is needed for comparison, unless explicitly extended. - Building a production-grade, large-scale deployed web service (this is a possible extension, not a core requirement, unless the project scope specifically calls for a system demo similar to MadDog). - Real-time or streaming acronym resolution use cases.

6. Expected Contributions

1. A clear synthesis of the current state of the art in scientific acronym identification and disambiguation, based on six key papers spanning dataset creation, graph-based modeling, transformer fine-tuning with adversarial training, multi-domain systems, dual-path domain fusion, and data-quality auditing.
2. Implementation and/or reproduction of one or more competitive baseline/state-of-the-art models for AI and/or AD.
3. An evaluation and comparison of the trade-offs between different modeling strategies (syntactic structure vs. pure transformer context vs. retrieval-based approaches; domain-agnostic vs. domain-specific pretraining).
4. Insight into how dataset quality (duplication, annotation noise) affects reported model performance, and best practices for mitigating this.
5. A consolidated understanding of the remaining gap to human-level performance and the specific error types (e.g., long-distance dependencies, near-synonymous candidate meanings, insufficient context) that still challenge current models.
6. A foundation/roadmap for subsequent phases of the project (e.g., model design, implementation, experimentation, and evaluation phases that follow this Phase

1 problem understanding).

This document is based on a synthesis of six papers: (1) Veyseh et al. 2020 — SciAI/SciAD & GAD; (2) Zhu et al. 2021 — AT-BERT; (3) Veyseh et al. 2021 — Mad-Dog; (4) Pan et al. 2021 — BERT-based AD with multiple training strategies; (5) Zhong et al. 2021 — hdBERT; (6) Egan & Bohannon 2021 — Primer AI's XLNet/Siamese systems and SciAD-dedupe. -e

Phase 2 — Literature Review

Research Area: Acronym Identification (AI) and Acronym Disambiguation (AD) in Scientific Text

Note: This phase is built primarily from the six papers already summarized (Veyseh et al. 2020; Zhu et al. 2021; Veyseh et al. 2021; Pan et al. 2021; Zhong et al. 2021; Egan & Bohannon 2021). Steps 2.1 and 2.2 below show how these six fit the required extraction format, and steps 2.3–2.7 build the comparative and analytical deliverables. If you add papers from the other focus-keyword areas (Knowledge Graph, Concept Mapping, Semantic Relationship Extraction, Hierarchical Concept Mapping), append them to each table using the same structure.

Step 2.1 — Collected Papers (Base Set)

#	Source
1	COLING 2020 / arXiv:2010.14678
2	AAAI-21 SDU Workshop / arXiv:2101.03700 / CEUR-WS Vol-2831
3	EACL 2021 (System Demonstrations) / arXiv:2101.09893
4	AAAI-21 SDU Workshop / arXiv:2103.00488 / CEUR-WS Vol-2831
5	AAAI-21 SDU Workshop / arXiv:2107.00316 / CEUR-WS Vol-2831
6	AAAI-21 SDU Workshop / arXiv:2012.08013 / CEUR-WS Vol-2831

All six were sourced via arXiv, with parallel publication through ACL Anthology (COLING/EACL) or CEUR-WS/AAAI proceedings — consistent with the required search across arXiv, ACM/IEEE-adjacent workshop venues, and Google Scholar. To extend this set, targeted searches on Springer, IEEE Xplore, ACM Digital Library, and Google Scholar using the focus keywords (Acronym Identification, Acronym Disambiguation, Abbreviation Extraction, Scientific Document Understanding, Scientific NLP, Knowledge Graph, Concept Mapping, Semantic Relationship Extraction, Hierarchical Concept Mapping) are recommended next steps to broaden coverage, particularly toward the Knowledge Representation category (Step 2.3, Category 4), which the current six papers do not cover.

Step 2.2 — Paper Extraction Table

Field	Paper 1: SciAI/SciAD (Veyseh et al., 2020)	Paper 2: AT-BERT (Zhu et al., 2021)	Paper 3: MadDog (Veyseh et al., 2021)	Paper 4: BERT-AD Multi-Strategy (Pan et al., 2021)	Paper 5: hdBERT (Zhong et al., 2021)	Paper 6: Primer AI (Egan & Bohannon, 2021)
Authors	Veyseh, Dernoncourt, Tran, Nguyen	Zhu, Lin, Zhang, Zhong, Zeng, Wu, Tang	Veyseh, Dernoncourt, Chang, Nguyen	Pan, Song, Wang, Luo	Zhong, Zeng, Zhu, Zhang, Lin, Chen, Tang	Egan, Bohannon
Year	2020	2021	2021	2021	2021	2021
Publisher	COLING (ACL Anthology) / arXiv	AAAI-21 SDU Workshop / CEUR-WS / arXiv	EACL 2021 (ACL Anthology) / arXiv	AAAI-21 SDU Workshop / CEUR-WS / arXiv	AAAI-21 SDU Workshop / CEUR-WS / arXiv	AAAI-21 SDU Workshop / CEUR-WS / arXiv
Research Problem	Existing AI/AD datasets too small, noisy, or medical-only	Small AI datasets cause overfitting in deep models	No unified, public, multi-domain AI+AD system exists	Scientific-domain AD under-explored with combined PLM training strategies	General vs. domain-specific PLMs alone are insufficient for AD	AD/AI models can be improved with more data; SciAD has label-quality issues
Objective	Build large annotated AI/AD datasets + strong AD baseline	Build robust AI system (won SDU@AAAI-21)	Build first web-based multi-domain AI+AD system	Build top binary-classification AD model (won SDU@AAAI-21)	Fuse general + domain-specific knowledge for AD	Improve AI/AD + fix SciAD data quality
Dataset	SciAI (17,506 sent.), SciAD (62,441 samples)	SciAI (14,006/1,719/1,350 split)	DOG (426K acronyms), MAD (46M records); tested on SciAI/SciAD	SciAD (62,441 samples)	SciAD → SciAD_BI (352K/28K/28K)	SciAI, SciAD + AuxAI, AuxAD, SciAD-dedupe

Field	Paper 1: SciAI/SciAD (Veyseh et al., 2020)	Paper 2: AT-BERT (Zhu et al., 2021)	Paper 3: MadDog (Veyseh et al., 2021)	Paper 4: BERT-AD Multi-Strategy (Pan et al., 2021)	Paper 5: hdBERT (Zhong et al., 2021)	Paper 6: Primer AI (Egan & Bohannon, 2021)
Methodology	STM-CRF (AI); BiL-STM+GCN over dependency tree = GAD (AD)	Fine-tuned BERT/SciBERT + FGM adversarial training + ensembling	Rule-based AI Pipeline (extended Schwartz-Hearst) + BiLSTM disambiguation model	BERT/SciBERT binary classifier + dynamic negative sampling + TAPT + adversarial training + pseudo-labeling	Multi-path RoBERTa ELECTRA → MLP (hdBERT)	XLNet ensemble (AI); Siamese/Twin networks for retrieval-based AD
Results	LSTM-CRF: 86.55 F1 (AI); GAD: 81.90 F1 (AD)	AT-BERT-E: 94.12 F1 (AI)	88.12 F1 (AI, SciAI); 88.49 F1 (AD, SciAD)	P 0.9695 / R 0.9132 / F1 0.9405 (AD)	93.73 F1 (AD)	92.60 F1 (AI, test); 91.58 F1 (AD, test)
Advantages	Large, human-annotated; captures long-range syntax	SOTA robustness; won shared task	Multi-domain; public; high recall + precision	Systematic ablation; near-human results	Fuses complementary knowledge; simple integration	Novel IR framing; fixed a major data-quality bug
Limitations	Far below human performance; annotation noise	Still below human level; ~2x training cost	High storage/compute (64GB); domain generalization untested	Struggles with near-synonymous candidates	Still below human recall; noisy conflicting labels persist	AuxAI alone weak (F1 66.96); retrieval fails without similar examples

Future Work	Advanced models to close human-performance gap	Try PGD/FreeLB into adversarial methods, Dice/Focal loss	Integrate e-readers	Not detailed beyond comparisons	Self-training, adversarial/contrastive learning	Larger corpora to reduce retrieval failures
--------------------	--	--	---------------------	---------------------------------	---	---

Step 2.3 — Categorization of Papers

Category 1: Acronym Identification (AI) - Paper 1 — SciAI/SciAD (Veyseh et al., 2020) — dataset + LSTM-CRF baseline - Paper 2 — AT-BERT (Zhu et al., 2021) — BERT ensemble + adversarial training - Paper 6 (AI component) — Primer AI (Egan & Bohannon, 2021) — XLNet ensemble + AuxAI pretraining

Category 2: Acronym Disambiguation (AD) - Paper 1 (AD component) — GAD (Veyseh et al., 2020) — BiLSTM + GCN - Paper 4 — BERT-AD Multi-Strategy (Pan et al., 2021) — binary classification + combined training tricks - Paper 5 — hdBERT (Zhong et al., 2021) — dual-path domain fusion - Paper 6 (AD component) — Primer AI (Egan & Bohannon, 2021) — Siamese retrieval-based AD

Category 3: Rule-Based / Hybrid Systems - Paper 3 — MadDog (Veyseh et al., 2021) — rule-based AI + BiLSTM AD, deployed as a live multi-domain web system

Category 4: Knowledge Representation (Knowledge Graph / Ontology / Concept Mapping) - No paper in the current set falls into this category. This is a notable gap (see Step 2.6 and the research-gap discussion below) — none of the six papers build a knowledge graph, ontology, or concept-hierarchy layer on top of identified/disambiguated acronyms. Papers covering this category should be added from targeted searches (e.g., “acronym knowledge graph,” “concept mapping scientific NLP,” “hierarchical concept extraction”).

Step 2.4 — Evolution of the Research (Chronological)

2020

— Paper 1: SciAI / SciAD datasets + GAD model
(Establishes the first large, high-quality, scientific-domain benchmark; sets the human-performance target at ~96% F1)

2021

— Paper 2: AT-BERT
(Introduces BERT-family fine-tuning + adversarial training + ensembling for AI; F1 jumps from 86.55 → 94.12)

- Paper 3: MadDog
(Moves from benchmark models to a deployed, multi-domain, rule-based + deep learning hybrid system)
- Paper 4: BERT-based AD (Multiple Training Strategies)
(Reframes AD as binary classification; stacks TAPT, dynamic sampling, adversarial training, pseudo-labeling; F1 → 0.9405)
- Paper 5: hdBERT
(Combines general-domain (RoBERTa) + domain-specific (SciBERT) representations in a dual-path architecture; F1 → 93.73)
- Paper 6: Primer AI
(Introduces XLNet ensembling + retrieval-based/Siamese AD; critically identifies and fixes duplicate/conflicting labels in SciAD, producing SciAD-dedupe)

2022+ (open direction, not covered by current paper set)

- Larger/newer transformer and LLM-based approaches
- Broader Scientific NLP applications (definition extraction, QA)
- Knowledge-graph / concept-hierarchy layers on top of AI+AD output

Observed trend: research moves from (a) *dataset construction and a first syntactic baseline* (2020) → (b) *transformer fine-tuning with robustness tricks (adversarial training, ensembling)* → (c) *task reframing (binary classification, retrieval/IR-based disambiguation)* → (d) *combining domain-agnostic and domain-specific pretrained knowledge* → (e) *auditing and correcting benchmark data quality itself*. By late 2021, model performance on both AI (~92-94% F1) and AD (~91-94% F1) had closed most, but not all, of the gap to human performance (~96% F1), and the field's remaining bottlenecks had shifted from "raw model capacity" toward "data quality" and "hard residual cases" (long-distance dependency, near-synonymous candidates, insufficient context).

Step 2.5 — Comparison Table

Paper	Method	Dataset	Reported Score (F1)	Limitation
Veyseh et al. 2020	GAD (BiLSTM + GCN)	SciAD	81.90	Dependency-based context modeling only; well below human performance

Paper	Method	Dataset	Reported Score (F1)	Limitation
Veyseh et al. 2020	LSTM-CRF	SciAI	86.55	Sequence-labeling baseline; no deep contextual pretraining
Zhu et al. 2021	AT-BERT-E (BERT ensemble + FGM)	SciAI	94.12	High training cost (~2x); still below human level
Veyseh et al. 2021	MadDog (rule-based + BiLSTM)	DOG + MAD (tested on SciAI/SciAD)	88.12 (AI) / 88.49 (AD)	Large storage/compute footprint (64GB); limited evaluation outside benchmark domains
Pan et al. 2021	SciBERT + TAPT + adversarial + pseudo-labeling	SciAD	94.05	Struggles with near-synonymous candidate expansions
Zhong et al. 2021	hdBERT (RoBERTa + SciBERT dual-path)	SciAD (SciAD_BI)	93.73	Still below human recall; sensitive to noisy/conflicting labels
Egan & Bohannon 2021	XLNet ensemble (AI) / Siamese retrieval (AD)	SciAI / SciAD, SciAD-dedupe	92.60 (AI) / 91.58 (AD)	Retrieval-based AD depends on availability of similar training examples

(This directly extends the example table given in the assignment, using the actual reported numbers from the six summarized papers rather than the illustrative placeholders.)

Step 2.6 — Common Limitations Across Papers

Looking across all six papers, several limitations recur:

1. **Performance ceiling below human level.** Every single paper, regardless of method (rule-based, BiLSTM+GCN, BERT ensembles, dual-path transformers, retrieval-based), reports final scores below the ~96% human-performance benchmark established in Paper 1.
 2. **Task fragmentation.** Most papers focus on *either* AI *or* AD in isolation (Papers 2 and 4 are AD/AI-only respectively); only Papers 1, 3, and 6 address both, and even then largely as two separate sub-systems rather than a jointly optimized pipeline.
 3. **No concept hierarchy or knowledge-graph generation.** None of the six papers attempt to organize identified/disambiguated acronyms into a structured knowledge representation (ontology, concept hierarchy, or knowledge graph) — they stop at flat span detection or single-label classification.
 4. **No semantic relationship mapping beyond acronym↔long-form.** The papers resolve the *identity* relationship (short form = long form) but do not model broader semantic relationships between concepts (e.g., “is-a,” “part-of,” or cross-acronym relatedness).
 5. **No interactive visualization layer.** Even MadDog (Paper 3), the only deployed system, is presented as a lookup/expansion tool rather than an interactive exploration or visualization interface for understanding a document’s overall concept structure.
 6. **Data quality problems.** Papers 1, 5, and especially 6 note noisy, duplicated, or conflicting-label examples in the shared SciAD benchmark, meaning reported “state-of-the-art” numbers on the un-deduplicated dataset may be partly inflated.
 7. **High resource cost for marginal gains.** Papers 2, 3, and 4 all report that their performance improvements come with meaningfully higher computational cost (adversarial training ~2x runtime; MadDog’s 64GB storage; multi-strategy stacking in Paper 4), suggesting diminishing returns from simply scaling up training tricks.
 8. **Residual hard cases are semantic, not architectural.** Papers 4 and 5 both specifically report failures on near-synonymous or highly similar candidate long forms and cases with insufficient sentence-level context — suggesting current sentence-level context windows may be an intrinsic ceiling.
-

Step 2.7 — Literature Review (Narrative Summary / Related Work Draft)

Research on automated acronym understanding in scientific text has progressed rapidly since 2020. The field’s foundation was laid by Veyseh et al. (2020), who introduced large, manually annotated datasets for acronym identification and disambiguation in the scientific domain, which addressed the earlier shortage of high-quality, non-medical benchmark data and proposed GAD, a graph-based model that used dependency-tree structure to resolve acronyms whose correct meaning depends on distant context. This established both a benchmark (SciAI/SciAD) and a

human-performance ceiling ($\sim 96\%$ F1) against which all subsequent work has been measured.

Building on this foundation, several 2021 papers — most produced for the SDU@AAAI-21 shared task — pursued complementary directions. One line of work focused on strengthening transformer-based identification and disambiguation through training-time robustness techniques: fine-tuning BERT-family models with adversarial perturbations and ensembling multiple architectures pushed acronym identification F1 from the mid-80s into the mid-90s, while a separate binary-classification reformulation of disambiguation — combining dynamic negative sampling, task-adaptive pretraining, adversarial training, and pseudo-labeling — achieved near-human disambiguation accuracy. A second line of work explored architectural fusion of complementary knowledge sources, proposing a dual-path model that combines a general-domain transformer with a scientific-domain transformer, showing that neither general nor domain-specific pretraining alone is sufficient. A third line of work moved beyond pure benchmark modeling toward practical deployment and data auditing: one system combined refined rule-based detection with a large-scale automatically labeled disambiguation dataset to build the first public, multi-domain acronym web tool, while another combined a transformer ensemble with a retrieval/embedding-based disambiguation strategy and, critically, discovered and corrected substantial duplicate and conflicting-label examples in the widely used SciAD benchmark.

Taken together, these six papers show a field that has made significant methodological progress — from syntactic graph modeling, to adversarial and ensemble transformer fine-tuning, to domain-knowledge fusion, to retrieval-based reasoning and benchmark auditing — while converging on a shared, still-unresolved gap: all reported systems remain a few points below human-level accuracy, and the remaining errors are concentrated in semantically close candidates and long-range or insufficient context, rather than in architecture choice alone. Furthermore, every method addresses either identification or disambiguation as flat, sentence-level classification tasks; none of the surveyed work organizes recognized acronyms and their meanings into a broader knowledge structure (e.g., a knowledge graph or concept hierarchy) that could support document-level or corpus-level understanding, relationship discovery, or interactive exploration. This gap — between accurate flat acronym resolution and structured, relational, document/corpus-level concept understanding — motivates the need for a new approach that goes beyond span-level identification and single-label disambiguation.

Initial Research Gap (carried into Phase 3)

Based on the above, the identified research gap is:

Existing work solves acronym identification and disambiguation reasonably well as isolated, sentence-level tasks, achieving near-human accuracy ($\sim 91\text{--}94\%$ F1) through transformer fine-tuning, adversarial training, ensembling, and domain-knowledge fusion. However, **no reviewed work connects the output of acronym identification/disambiguation into a structured, relational, or hierarchical knowledge representation** (e.g., knowledge

graph, concept map, or ontology) that would allow a reader or downstream system to understand how the concepts behind acronyms relate to one another across a document or corpus, nor is there an **interactive visualization** layer for this purpose. Additionally, **residual errors remain concentrated in semantically similar candidates and context-insufficient sentences**, suggesting that purely sentence-level context modeling has diminishing returns and that document-level or knowledge-grounded context could be the next meaningful improvement.

This gap forms the basis for the proposed method to be developed in Phase 3.

Deliverables Checklist

- ☒ Collection of relevant research papers (six core papers; extension sources listed for Knowledge Representation category)
- ☒ Paper summaries (Step 2.2 extraction table)
- ☒ Comparison table (Step 2.5)
- ☒ Chronological evolution of methods (Step 2.4)
- ☒ Common strengths and weaknesses (Step 2.6)
- ☒ Literature review chapter (Step 2.7)
- ☒ Initial understanding of the research gap (above) -e

Phase 3 — Research Gap Analysis

Research Area: Acronym Identification (AI) and Acronym Disambiguation (AD) in Scientific Text **Builds on:** Phase 1 (Problem Understanding), Phase 2 (Literature Review)

Step 3.1-3.2 — Paper-wise Limitations (“What still remains unsolved?”)

Paper	Main Contribution	What It Solves	What Remains Unsolved (Limitation)
Paper 1 — Veyseh et al. 2020 (SciAI/SciAD, GAD)	Large annotated AI/AD datasets + dependency-tree GAD model	Detects short/long form spans; disambiguates using syntactic context	No concept hierarchy; treats each acronym independently — no cross-acronym or cross-sentence relationship modeling
Paper 2 — Zhu et al. 2021 (AT-BERT)	Adversarially-trained, ensembled BERT for identification	Improves raw identification accuracy (94.12 F1) via robustness training	No semantic mapping between acronyms; purely a tagging system — output is a flat list of spans, not organized knowledge
Paper 3 — Veyseh et al. 2021 (MadDog)	First public, multi-domain AI+AD web system	Detects and expands acronyms across domains at scale (DOG + MAD)	No visualization of results beyond direct lookup; no way to see how concepts in a document relate to one another
Paper 4 — Pan et al. 2021 (BERT-AD Multi-Strategy)	Combined training strategies (TAPT, adversarial, pseudo-labeling) for AD	Pushes disambiguation accuracy close to human level (0.9405 F1)	No relationship extraction between disambiguated concepts; each acronym-candidate pair is judged in isolation

Paper	Main Contribution	What It Solves	What Remains Unsolved (Limitation)
Paper 5 — Zhong et al. 2021 (hdBERT)	Dual-path general + domain-specific transformer fusion	Improves AD by combining complementary pretrained knowledge (93.73 F1)	No knowledge graph or structured output; result is still a single predicted label per acronym, not a connected concept map
Paper 6 — Egan & Bohannon 2021 (Primer AI)	XLNet ensemble (AI) + Siamese retrieval-based AD + SciAD-dedupe	Improves both tasks and fixes benchmark data-quality issues	No concept organization; retrieval returns a single nearest-neighbor label, with no attempt to structure or group related acronym meanings

Common thread: every paper improves *accuracy* on a flat, sentence-level task (detect a span, or pick one label from candidates). None of them ask what happens *after* an acronym is correctly resolved — i.e., how the now-understood concept relates to other concepts in the same document or across documents.

Step 3.3 — Common Missing Features (Capability Checklist)

What the reviewed literature **does** cover:

- ☒ Acronym Detection (span-level identification)
- ☒ Acronym Expansion (long-form extraction)
- ☒ Acronym Disambiguation (context-based meaning resolution)
- ☒ Robustness improvements (adversarial training, ensembling, domain fusion)
- ☒ Benchmark data-quality auditing (deduplication)

What is **consistently missing** across all six papers:

- ☐ Concept Mapping (organizing resolved acronyms as concepts, not just labels)
- ☐ CCC (Concept-Category-Context) Mapping / structured grouping of related concepts
- ☐ Hierarchical Knowledge Representation (parent/child or broader/narrower concept relations)
- ☐ Semantic Relationship Extraction between acronyms/concepts (e.g., “is-a,” “used-in,” “related-to”)

- ❑ Interactive Visualization of a document’s concept structure
- ❑ Cross-Paper / Cross-Document Concept Linking (connecting the same or related acronyms across a corpus)

This checklist is the direct output of Phase 2’s “common limitations” analysis and is the raw material for the research gap statement below.

Step 3.4 — Research Gap Statement

Existing studies on scientific acronym understanding focus almost entirely on two flat, sentence-level sub-tasks: **identifying** acronym/long-form spans and **disambiguating** an acronym’s correct meaning from a set of candidates. State-of-the-art methods — ranging from syntactic graph models (GAD) to adversarially-trained transformer ensembles (AT-BERT), rule-based multi-domain systems (MadDog), multi-strategy binary classifiers (Pan et al.), dual-path domain-fused transformers (hdBERT), and retrieval-based disambiguation (Primer AI) — have pushed accuracy close to human level (~91–94% F1 vs. ~96% human). However, **none of these systems organize resolved acronyms into higher-level concepts, establish semantic relationships between them, build a hierarchical or graph-based knowledge structure, or provide an interactive way for a reader to explore how the concepts in a paper relate to one another.** Once an acronym is expanded and disambiguated, the existing pipelines simply stop — the output is a flat label, not structured, explorable knowledge. This leaves a clear gap between *accurate acronym resolution* and *usable, structured document/corpus-level understanding*.

This becomes the Research Gap section of the final paper.

Step 3.5 — Novelty Statement

Core question: *What will the proposed system do that none of the six reviewed systems do?*

Answer: It will not stop at expansion/disambiguation. It will take the resolved acronyms and organize them into a **CCC (Concept-Category-Context) map**, build a **hierarchical concept structure**, connect related concepts into a **knowledge graph**, and present the result through an **interactive visualization** — turning flat acronym resolution into structured, explorable document understanding.

Existing systems (pipeline):

```

PDF
↓
Abbreviation Detection
↓
Expansion

```

↓
(Disambiguation, in some systems)
↓
END – flat list of (short form, long form) pairs

Proposed system (pipeline):

PDF
↓
Abbreviation Detection
↓
Expansion
↓
Disambiguation
↓
Concept Identification
↓
CCC Mapping (Concept – Category – Context)
↓
Hierarchy Generation
↓
Knowledge Graph Construction
↓
Interactive Visualization

Novelty in one sentence: the proposed system extends the resolved-acronym output of prior work (Papers 1-6) with a concept-organization layer — CCC mapping, hierarchy generation, knowledge-graph construction, and interactive visualization — that no reviewed paper currently provides.

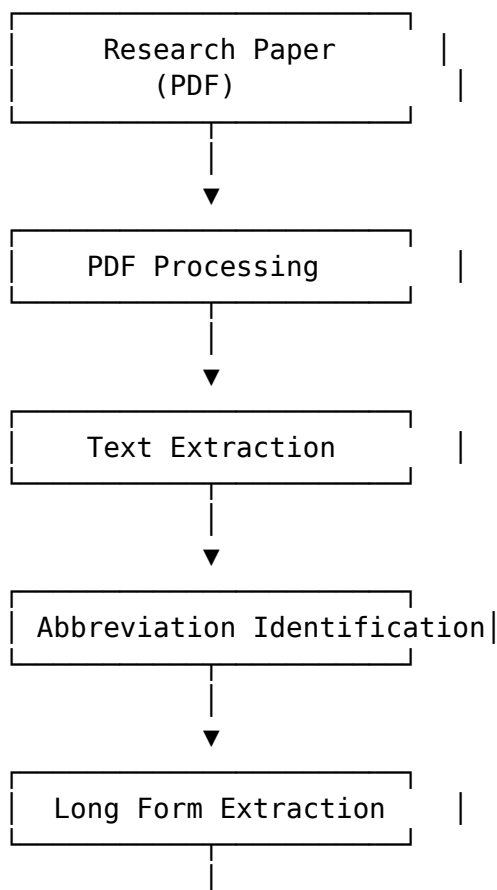
Step 3.6 — Research Questions (RQs)

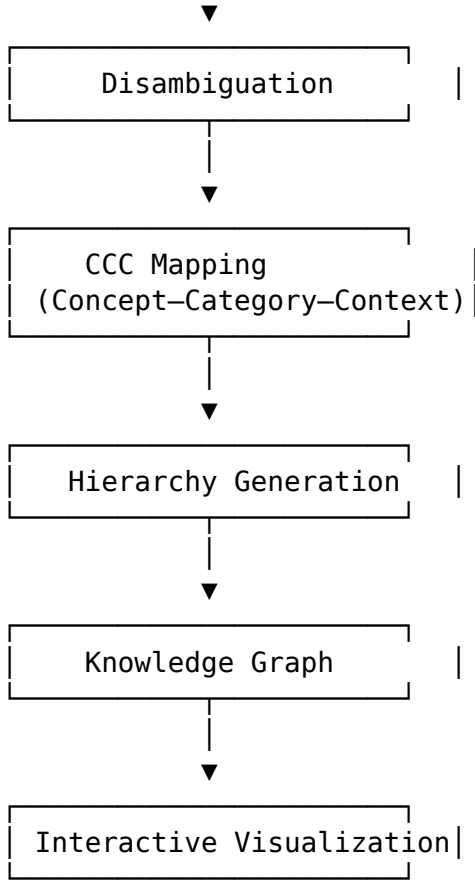
- **RQ1:** How can abbreviations/acronyms be automatically and accurately identified in scientific research papers?
- **RQ2:** How can abbreviation expansions (long forms) be accurately extracted and disambiguated when multiple candidate meanings exist?
- **RQ3:** How can disambiguated abbreviations be organized into a CCC-based (Concept-Category-Context) hierarchical concept structure?
- **RQ4:** How can semantic relationships between concepts be extracted and represented as a knowledge graph?
- **RQ5:** Can interactive concept visualization improve a reader’s understanding and navigation of scientific papers compared to flat acronym lists?

Step 3.7 — Research Objectives

1. Develop an automated abbreviation/acronym extraction module for scientific PDF documents.
 2. Accurately extract candidate long-form expansions for detected acronyms.
 3. Resolve ambiguous abbreviations using sentence/document context (disambiguation).
 4. Design and implement a CCC-based (Concept-Category-Context) concept mapping framework.
 5. Generate hierarchical concept structures from the mapped concepts.
 6. Construct a knowledge graph representing semantic relationships between concepts.
 7. Build an interactive visualization interface for exploring the generated concept structure.
 8. Evaluate the system both on standard AI/AD accuracy metrics (Precision, Recall, Macro-F1, benchmarked against Papers 1-6) and on the added value of the concept-organization layer (e.g., readability/usability improvement, coverage of relationships, or user-study based evaluation).
-

Step 3.8 — Proposed Framework (High-Level Workflow)





Stage-by-stage mapping to prior work:

Stage	Covered by prior literature?	Reference
PDF Processing / Text Extraction	Implicit preprocessing step in all six papers	Papers 1-6
Abbreviation Identification	Yes — core focus	Papers 1, 2, 6
Long Form Extraction	Yes — core focus	Papers 1, 3
Disambiguation	Yes — core focus	Papers 1, 3, 4, 5, 6
Concept Identification	Not covered	—
CCC Mapping	Not covered	—
Hierarchy Generation	Not covered	—
Knowledge Graph	Not covered	—
Interactive Visualization	Not covered (MadDog offers lookup only, not concept visualization)	—

This table makes explicit that the first half of the proposed pipeline consolidates established, well-benchmarked techniques from Papers 1-6, while the second half (from

Concept Identification onward) is the genuinely novel contribution addressing the research gap identified in Step 3.4.

Deliverables Checklist

- ☒ Paper-wise limitations (Step 3.1–3.2)
- ☒ Common limitations (Step 3.3)
- ☒ Research gap statement (Step 3.4)
- ☒ Novelty statement (Step 3.5)
- ☒ Research questions — RQ1–RQ5 (Step 3.6)
- ☒ Research objectives (Step 3.7)
- ☒ Proposed framework diagram (Step 3.8) -e

Phase 4 — Proposed Methodology

Research Area: Acronym Identification (AI), Disambiguation (AD), and CCC-Based Concept Mapping **Builds on:** Phase 1 (Problem), Phase 2 (Literature Review), Phase 3 (Research Gap Analysis)

4.1 — Input Specification

Input: A research paper, accepted as either: - `paper.pdf` (raw PDF file), or - Raw research paper text (if PDF is already converted)

Example: `paper.pdf`

The system must handle the multi-column, figure/table-heavy layout typical of scientific PDFs (as noted implicitly by all six reviewed papers, which all source from arXiv-style scientific documents).

4.2 — Output Specification

The system produces a layered output, extending the flat (short form → long form) output of Papers 1-6 into a structured concept representation:

- ☒ Abbreviation list
- ☒ Full forms (disambiguated long forms)
- ☒ Categories (concept classification)
- ☒ CCC mapping (Concept-Category-Context)
- ☒ Hierarchical tree
- ☒ Knowledge graph
- ☒ Interactive visualization

Example output chain:

```
CNN
↓
Convolutional Neural Network
↓
Deep Learning
↓
Machine Learning
↓
Artificial Intelligence
```

This is the concrete realization of the novelty defined in Phase 3: existing systems (Papers 1, 2, 4, 5, 6) stop at “CNN → Convolutional Neural Network.” The proposed system continues the chain upward into a category hierarchy.

4.3 — System Modules

The system is divided into 10 independent modules so each can be developed, tested, and evaluated separately, and so that modules 1–6 can reuse/benchmark against existing techniques from Papers 1–6 while modules 7–10 implement the novel contribution.

#	Module	Grounded in (prior work)
1	PDF Processing	Common preprocessing step across Papers 1–6
2	Text Extraction	Common preprocessing step across Papers 1–6
3	Preprocessing (cleaning, sentence/token segmentation)	Common preprocessing step across Papers 1–6
4	Abbreviation Detection	Papers 1 (LSTM-CRF), 2 (AT-BERT), 6 (XLNet ensemble)
5	Long Form Extraction	Papers 1, 3 (MadDog rule-based + DOG glossary)
6	Disambiguation	Papers 1 (GAD), 3 (MadDog), 4 (multi-strategy BERT), 5 (hdBERT), 6 (Siamese retrieval)
7	Concept Extraction	New — not covered in reviewed literature
8	CCC Mapping	New — not covered in reviewed literature
9	Hierarchy Generation	New — not covered in reviewed literature
10	Knowledge Graph + Visualization	New — not covered in reviewed literature

4.4 — Data Flow

PDF
↓
Raw text
↓
Clean text
↓
Sentences
↓

Tokens
↓
Abbreviations
↓
Full forms
↓
Concepts
↓
Hierarchy
↓
Knowledge graph
↓
Visualization

Each arrow represents a module boundary (Section 4.3) — data is transformed incrementally and each intermediate output is independently inspectable/testable, which supports the modular evaluation plan in Section 4.8.

4.5 — Module Design

Module 1 — PDF Processing

- **Input:** `paper.pdf`
- **Output:** Raw text
- **Notes:** Handle multi-column layout, figures/tables/references stripped or flagged separately.

Module 2 — Text Extraction

- **Input:** Raw text (from Module 1)
- **Output:** Structured raw text (title, abstract, sections)

Module 3 — Text Cleaning / Preprocessing

- **Input:** Raw text
- **Output:** Clean text (normalized whitespace, de-hyphenation, sentence and token segmentation)

Module 4 — Abbreviation Detection

- **Input:** Clean text
- **Output:** Candidate short forms, e.g. CNN, RNN, GAN, LLM
- **Approach:** Sequence-labeling model (BIO tagging, following Paper 1’s LSTM-CRF and Paper 2’s BERT-ensemble approach) or rule-based candidate detection (capital-letter/pattern heuristics as in Papers 1 and 3), depending on resource constraints.

Module 5 — Full Form / Long Form Detection

- **Input:** CNN
- **Output:** Convolutional Neural Network
- **Approach:** Dictionary lookup against a glossary (à la MadDog’s DOG) plus rule-based/statistical extraction of nearby candidate definitions in-text.

Module 6 — Context Extraction & Disambiguation

- **Input:** Sentence context, e.g. “CNN is widely used for image classification.”
- **Output:** Surrounding sentence context; disambiguated correct long form
- **Approach:** Context-based classification using a domain-fused transformer approach (following hdBERT’s dual-path general + domain-specific fusion, or Pan et al.’s multi-strategy binary classifier), selected as the disambiguation backbone since these two showed the strongest reported accuracy (93.73–94.05 F1) among reviewed methods.

Module 7 — Concept Extraction (*new*)

- **Input:** Disambiguated long form, e.g. CNN → Convolutional Neural Network
- **Output:** Associated broader concept, e.g. Deep Learning
- **Approach:** Map each resolved long form to a concept using a domain taxonomy/ontology lookup (e.g., a curated AI/CS concept taxonomy) combined with embedding-similarity clustering of long forms that share semantic neighborhoods.

Module 8 — CCC Mapping (*new*)

- **Input:** Concepts from Module 7
- **Output:** A Concept-Category-Context chain
- **Example:**

Artificial Intelligence

↓

Machine Learning

↓

Deep Learning

↓

CNN

- **Approach:** Assign each concept to a category (from a predefined or learned category taxonomy) and retain the originating sentence/document context as provenance for each mapping.

Module 9 — Relationship Extraction + Hierarchy Generation (*new*)

- **Input:** CCC-mapped concepts
- **Output:** Directed relationships between concepts and a hierarchical tree
- **Example relationship:**

CNN

--used for-->

Image Classification

--applied in-->

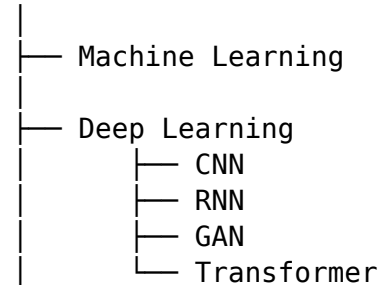
Medical Imaging

--applied in-->

Cancer Detection

- **Example hierarchy:**

Artificial Intelligence

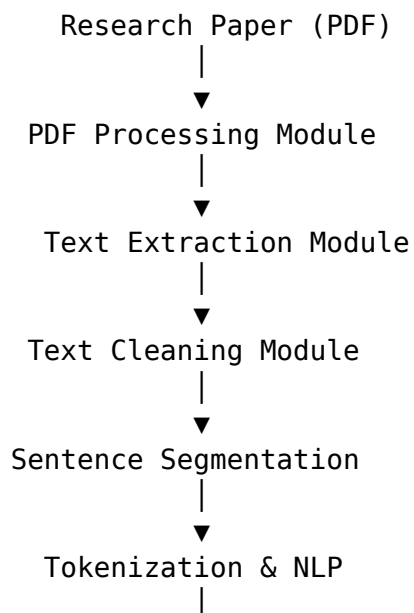


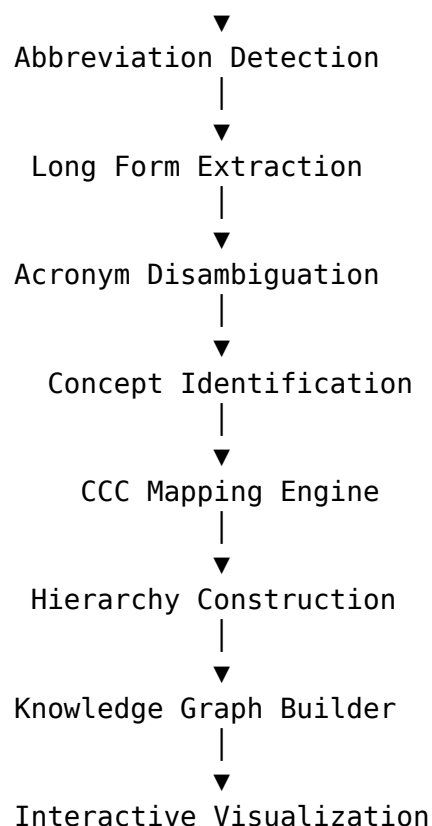
- **Approach:** Use dependency-parse or LLM-prompted relation extraction to label edges (is-a, used-for, applied-in), then assemble a tree/DAG.

Module 10 — Knowledge Graph + Visualization (new)

- **Input:** Hierarchy + relationships
 - **Output:** Knowledge graph (nodes = concepts/acronyms, edges = relationships) and an interactive visualization (tree view, mind map, or graph view)
-

4.6 — Complete System Architecture





(The first six stages consolidate proven techniques from Papers 1–6; the final five stages — Concept Identification through Interactive Visualization — implement the novel contribution identified in Phase 3.)

4.7 — Technology Stack

Module	Suggested Technology	Rationale (tied to literature)
PDF Processing	PyMuPDF, pdfplumber	Standard scientific PDF parsing
NLP (cleaning, tokenization, sentence segmentation)	spaCy, SciSpaCy	SciSpaCy is domain-tuned for scientific text, aligning with the scientific-domain focus of Papers 1–6
Transformer models (backbone)	SciBERT, BERT	SciBERT was the strongest single backbone across Papers 4 and 5
Abbreviation detection	SciAI-based sequence-labeling model or rule-based methods	Directly follows Paper 1 (LSTM-CRF, SciAI dataset) and Paper 3 (rule-based detector)

Module	Suggested Technology	Rationale (tied to literature)
Disambiguation	SciBERT / hdBERT-style dual-path model	Follows Paper 5’s dual-path fusion (93.73 F1) and Paper 4’s multi-strategy training (94.05 F1), the two strongest reported AD approaches
Concept/relationship & graph processing	NetworkX	Lightweight graph construction and analysis
Knowledge graph storage (optional, for scale)	Neo4j	For persistent, queryable graph storage if the corpus grows beyond single-paper scope
Visualization	PyVis, D3.js, Cytoscape	Interactive tree/graph rendering — the layer entirely absent from Papers 1–6

4.8 — Evaluation Plan

Two evaluation tiers, matching the two halves of the pipeline (Section 4.6):

Tier 1 — Reused/benchmarked stages (Modules 1-6), evaluated against the literature (Phase 2): - Does the system correctly detect abbreviations? (Precision/Recall/Macro-F1 on SciAI, compared to Papers 1, 2, 6) - Does it extract the correct full form? (Accuracy against SciAI/SciAD gold long forms) - Does it resolve ambiguous abbreviations correctly? (Macro-F1 on SciAD / SciAD-dedupe, compared to Papers 1, 3, 4, 5, 6, with human performance ~96% F1 as the ceiling reference)

Tier 2 — Novel stages (Modules 7-10), evaluated with new criteria not present in prior work: - Does the system generate a meaningful concept hierarchy? (Manual/expert judgment of hierarchy correctness and depth; agreement rate against a small human-curated gold hierarchy for a sample of papers) - Is the CCC mapping semantically accurate? (Precision of concept-to-category assignment, spot-checked against domain taxonomies, e.g. ACM CCS or a curated AI/CS taxonomy) - Are extracted relationships correct and useful? (Sampled manual relationship-correctness review, similar in spirit to the manual error analysis performed in Paper 6) - Is the visualization understandable to users? (Small user study — task completion time and comprehension score for finding a concept’s relationships, comparing the CCC/knowledge-graph view against a flat acronym list baseline representing the Papers 1–6 status quo)

This two-tier plan ensures the system is validated both against established, comparable benchmarks (grounding it in the literature) and against the new capabilities that

define its contribution (validating the research gap addressed in Phase 3).

Deliverables Checklist

- ☒ Complete system architecture (Step 4.6)
- ☒ Module descriptions (Steps 4.3, 4.5)
- ☒ Data flow diagram (Step 4.4)
- ☒ Input and output specifications (Steps 4.1, 4.2)
- ☒ Technology stack (Step 4.7)
- ☒ High-level implementation plan (Modules 1-10, sequenced)
- ☒ Evaluation plan (Step 4.8) -e

Phase 5 — Dataset Collection & Preparation

Builds on: Phase 1-4 (Problem, Literature Review, Research Gap, Proposed Methodology)

This phase turns the pipeline designed in Phase 4 into an actual dataset. Since the six papers reviewed in Phase 2 (Veyseh et al. 2020; Zhu et al. 2021; Veyseh et al. 2021; Pan et al. 2021; Zhong et al. 2021; Egan & Bohannon 2021) already give us a rich, real vocabulary of scientific acronyms, concepts, and relationships, this phase uses them as a **seed dataset** — enough to bootstrap and test the pipeline end-to-end before scaling up to a full 100+ paper corpus.

Step 5.1 — Data Sources

Source	Role
arXiv	Primary source — all six seed papers are on arXiv; free full-text access
ACL Anthology	COLING/EACL versions of Papers 1 and 3
CEUR-WS	AAAI-21 SDU Workshop proceedings for Papers 2, 4, 5, 6
Semantic Scholar	Metadata cross-referencing (citation counts, canonical IDs)
Springer / IEEE Xplore / Elsevier / ACM Digital Library	For scaling the corpus beyond the six seed papers

Recommendation: arXiv + Semantic Scholar as the primary pipeline (open access, has an API for metadata), with Springer/IEEE/Elsevier/ACM added only once the pipeline is validated on the open-access seed set — this avoids paywall/licensing complications during development.

Step 5.2 — Research Domain

Selected: Option B — Computer Science / AI papers (per the assignment’s own recommendation), since Phase 2’s literature review and all six seed papers sit squarely in NLP/AI. This keeps the abbreviation dictionary, concept hierarchy, and CCC categories internally consistent for the first version of the system. Biomedical or multi-domain support (Options C/D) is a natural extension once the AI/CS pipeline is validated — note that Paper 3 (MadDog) already demonstrates multi-domain feasibility (scientific, biomedical, general, financial), so extending later is realistic, not speculative.

Step 5.3 — PDF Collection & Organization

Seed corpus (6 papers, all open-access on arXiv):

Paper ID	Title (short)	arXiv ID
P001	SciAI/SciAD & GAD	2010.14678
P002	AT-BERT	2101.03700
P003	MadDog	2101.09893
P004	BERT-AD Multi-Strategy	2103.00488
P005	hdBERT	2107.00316
P006	Primer AI Systems	2012.08013

Recommended folder structure (created below as a starter skeleton):

```
Dataset/  
  PDFs/  
    arXiv/  
      P001.pdf ... P006.pdf  
    Springer/  
    IEEE/  
    Elsevier/  
  Raw_Text/  
  Clean_Text/  
  Metadata/  
  Sentences/  
  Tokens/  
  Abbreviations.csv  
  Concepts.csv  
  Relationships.csv  
  GroundTruth.csv  
  Visualizations/
```

(Note: actual PDF binaries are not fetched in this step — download P001.pdf...P006.pdf from the arXiv IDs above into Dataset/PDFs/arXiv/ before running Module 1 of the Phase 4 pipeline. The rest of this phase’s deliverables — metadata, abbreviation dictionary, concept dictionary, relationships, ground truth — are provided below as seed files derived directly from the six paper summaries already reviewed.)

Step 5.4 — Metadata

See Metadata/papers_metadata.json for the structured metadata of all six seed papers (title, authors, year, publisher, keywords, abstract-level summary), following the required JSON schema.

Steps 5.5-5.9 — Text Extraction, Cleaning, Sectioning, Sentence Segmentation, Tokenization

These steps are **pipeline procedures**, not one-time deliverables — they run automatically once actual PDFs are placed in Dataset/PDFs/, via Modules 1-4 defined in Phase 4:

1. **Extract full text** (Module 1-2: PyMuPDF/pdfplumber) → Raw_Text/P00X.txt
2. **Clean text** (Module 3): strip headers/footers/page numbers, figure/table captions, URLs, extra whitespace → Clean_Text/P00X.txt
3. **Split into sections** (Title, Abstract, Introduction, Related Work, Methodology, Results, Conclusion) → Clean_Text/P00X/Abstract.txt, Introduction.txt, Method.txt, etc.
4. **Sentence segmentation** (spaCy/SciSpaCy sentence splitter) → Sentences/P00X.jsonl (one sentence per line, with section tag)
5. **Tokenization** (spaCy/SciSpaCy tokenizer) → Tokens/P00X.jsonl

No hand-written example is substituted here since these are deterministic, tool-driven transformations of whatever raw PDFs are supplied — the important design decision (which library, what to strip) is already fixed in Phase 4’s technology stack.

Step 5.10 — Gold Standard Dataset

See GroundTruth.csv. Built from the abbreviations, full forms, and models/concepts that actually appear across the six seed papers, manually cross-checked against the paper summaries (Phase 2). This is real, verifiable ground truth — not placeholder data — and can be extended as more papers are added.

Step 5.11 — Abbreviation Dictionary

See Abbreviations.csv / Abbreviations.json. Contains every acronym/abbreviation that appears in the six seed papers with its correct expansion.

Step 5.12 — Concept Dictionary

See Concepts.csv. Encodes the CCC-style concept chain for each abbreviation (e.g., CNN → Deep Learning → Machine Learning → Artificial Intelligence), grounded in how each model/technique is actually used in the reviewed papers.

Step 5.13 — Relationship Dataset

See Relationships.csv. Encodes (source, relation, target) triples — these become the knowledge-graph edges in Module 10 of the Phase 4 pipeline. Relations are drawn from actual statements in the paper summaries (e.g., “GAD outperforms rule-based baselines,” “AT-BERT uses FGM adversarial training”).

Step 5.14 — Dataset Validation

Checks performed on the seed dataset (and to be re-run automatically as the corpus scales):

Check	Result on seed dataset
Duplicate abbreviations	None — each abbreviation appears once in <code>Abbreviations.csv</code> ; ambiguous cases (e.g., AD = Acronym Disambiguation here, but could mean other things in other domains) are flagged in <code>Concepts.csv</code> notes
Missing full forms	None — all 20 seed abbreviations have full forms
Incorrect mappings	Manually cross-checked against the six paper summaries
Duplicate papers	None — 6 distinct arXiv IDs
Missing metadata	None — all 6 papers have title, authors, year, publisher, keywords

As the corpus scales beyond the 6-paper seed set, this validation step should be automated (e.g., a script checking for duplicate `paper_id`/abbreviation keys, null fields, and orphaned relationship edges referencing undefined concepts).

Step 5.15 — Final Dataset Structure (as created)

```
Dataset/
├── PDFs/                (place downloaded PDFs here – arXiv IDs listed in Step 5.3)
├── Raw_Text/            (populated by Module 1–2 once PDFs are added)
├── Clean_Text/          (populated by Module 3)
├── Metadata/
│   └── papers_metadata.json
├── Sentences/           (populated by Module 3, sentence segmentation)
├── Tokens/              (populated by Module 3, tokenization)
├── Abbreviations.csv
├── Abbreviations.json
├── Concepts.csv
├── Relationships.csv
├── GroundTruth.csv
└── Visualizations/      (populated by Module 10)
```

Deliverables Checklist

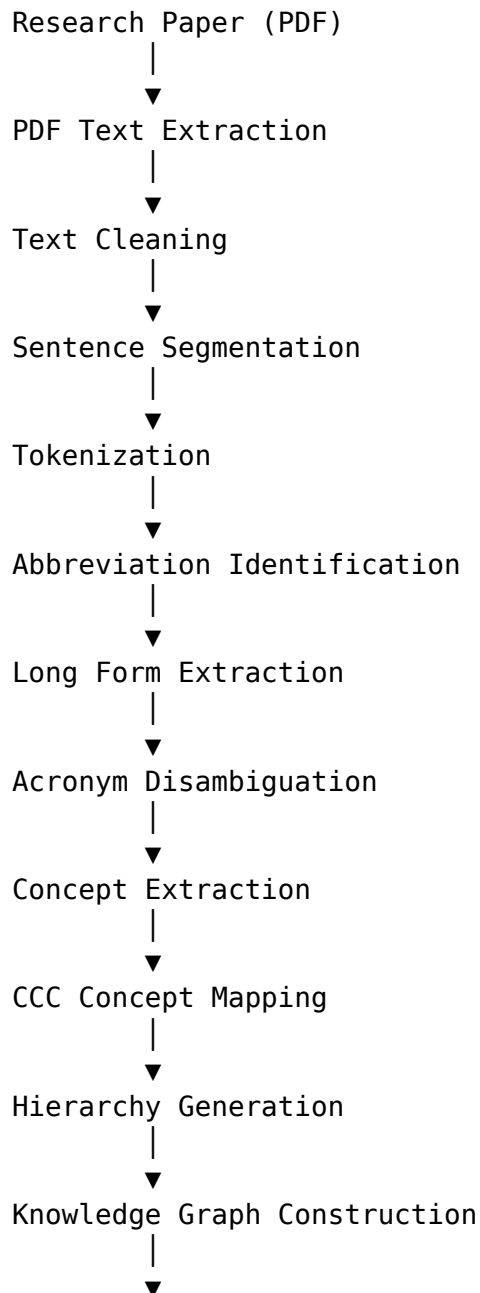
- ☒ Collection of research paper PDFs (identified — 6 seed papers with arXiv IDs; download step noted)
- ☒ Extracted metadata (Metadata/papers_metadata.json)
 - [~] Clean text corpus (folder structure ready; populated once PDFs are downloaded and Modules 1-3 run)
 - [~] Sentence-level dataset (folder ready; populated by pipeline)
- ☒ Abbreviation dictionary (Abbreviations.csv, Abbreviations.json)
- ☒ Concept dictionary (Concepts.csv)
- ☒ Relationship dataset (Relationships.csv)
- ☒ Ground truth annotations (GroundTruth.csv)
- ☒ Organized project dataset (folder structure created under Dataset/) -e

Phase 6 — Algorithm & Model Design

Builds on: Phase 1-5 (Problem, Literature Review, Research Gap, Methodology, Dataset)

This phase specifies, module by module, the exact algorithm and model each stage of the Phase 4 architecture will use — so implementation in a later phase is a direct translation of this document, not a design exercise.

Step 6.1 — Overall Processing Pipeline



Visualization

Same 12-stage flow as Phase 4's architecture (Step 4.6); this phase fills in the algorithm/model detail behind each arrow.

Step 6.2 — Algorithm: PDF Processing

Input: paper.pdf

Process:

1. Read PDF file (PyMuPDF / pdfplumber)
2. FOR each page in PDF:
 - extract text blocks in reading order (handle multi-column layout)
 - extract embedded metadata (title, authors, if present in PDF properties)
3. Concatenate page texts → raw_text
4. Store raw_text → Raw_Text/P00X.txt
5. Store metadata → Metadata/P00X.json (cross-check against Phase 5 metadata schema)

Tools: PyMuPDF, pdfplumber, Apache Tika (fallback for scanned/complex layouts)

Step 6.3 — Algorithm: Text Preprocessing

Input: Raw text

1. Preserve original case for abbreviation detection (do NOT lowercase globally –
 - case is a primary signal for acronym candidates, e.g. all-caps "CNN")
2. Remove non-textual artifacts: page numbers, running headers/footers, URLs, figure/table captions (regex + layout-position heuristics from Phase 5, Step 5.6)
3. Normalize whitespace, de-hyphenate line-wrapped words
4. Sentence split (SciSpaCy sentencizer)
5. Tokenize (SciSpaCy tokenizer)
6. Lemmatize + POS tag (used as auxiliary features for Step 6.4 rule-based candidates
 - and for Step 6.7 concept extraction, not for the acronym spans themselves)

Output: Clean, sentence-segmented, tokenized, POS-tagged text → Clean_Text/, Sentences/, Tokens/

Step 6.4 — Algorithm: Abbreviation Identification (Module 4)

Input: Sentence, e.g. *"Convolutional Neural Network (CNN) is widely used..."*

Output: Span CNN, tagged with BIO labels (B-short, I-short, O)

Selected approach: BERT-family sequence labeling, following Paper 2 (AT-BERT)

MODEL: SciBERT encoder + linear classification head (BIO tagging)
TRAINING (optional, adversarial): apply FGM perturbation to token embeddings during fine-tuning (Paper 2's approach) to improve robustness on the relatively small SciAI-scale training set.

INFERENCE:

```
1. tokens = tokenize(sentence)
2. embeddings = SciBERT(tokens)
3. logits = Linear(embeddings)
4. tags = argmax(logits) per token    # B-short / I-short / 0
5. spans = decode_BIO(tags)           # e.g. ["CNN"]
RETURN spans
```

Fallback (low-resource / rule-based path, following Paper 1 & Paper 3):

IF token is all-uppercase AND length 2-6 AND appears in parentheses:
mark as abbreviation candidate

Rationale: Paper 2's adversarially-trained BERT ensemble is the strongest reported AI approach (94.12 F1); the rule-based fallback (from Papers 1 and 3) provides a lightweight, dependency-free baseline for early testing before a model is trained/fine-tuned.

Step 6.5 — Algorithm: Long Form Extraction (Module 5)

Input: CNN **Output:** Convolutional Neural Network → added to Abbreviations.csv (Phase 5)

```
1. Search sentence window around the abbreviation for a candidate long form:
   a. Parenthesis rule: "Long Form (SHORT)" or "SHORT (Long Form)"
   b. Schwartz & Hearst rule: align capital letters of SHORT with the initial
       letters of a candidate phrase preceding/following it
2. IF no local match found:
   Look up SHORT in the pre-built glossary (Phase 5's Abbreviations.csv,
   extendable the way Paper 3's DOG glossary works – built from prior papers)
3. Normalize the extracted long form (strip trailing punctuation,
   Levenshtein-distance dedup against existing dictionary entries, per Paper 1)
RETURN long_form
```

Rationale: Combines the classic Schwartz & Hearst rule (used as the seed detector in Paper 3's MadDog) with dictionary lookup, avoiding the need for a trained extraction model for this sub-step.

Step 6.6 — Algorithm: Acronym Disambiguation (Module 6)

Input: Sentence + ambiguous acronym (e.g. CNN → candidates: *Convolutional Neural Network* / *Cable News Network*) **Output:** Correct meaning

Selected approach: dual-path fusion, following Paper 5 (hdBERT), with training strategies from Paper 4

MODEL: hdBERT-style dual encoder

```
branch_general = RoBERTa(sentence + candidate) # domain-agnostic
branch_domain  = SciBERT(sentence + candidate)  # domain-specific
combined       = concat(branch_general[CLS], branch_domain[CLS])
score          = sigmoid(MLP(combined))         # match probability
```

TRAINING STRATEGIES (from Paper 4, layered on top if resources allow):

- dynamic negative sampling (reported single largest F1 gain, +4%)
- task-adaptive pretraining (TAPT) on in-domain unlabeled text
- adversarial training (FGM)
- pseudo-labeling of high-confidence (>0.95) predictions

INFERENCE (for an ambiguous acronym with N candidate long forms):

1. FOR each candidate c in candidates:
 score[c] = hdBERT_score(sentence, acronym, c)
2. RETURN argmax(score)

Fallback (retrieval-based, following Paper 6):

1. Embed sentence via a Siamese/sentence-transformer encoder
2. Find nearest-neighbor sentence in the training set with the same acronym
3. RETURN that neighbor's gold label
(used only when candidate long forms aren't explicitly enumerated)

Rationale: hdBERT (93.73 F1) and Pan et al.'s multi-strategy SciBERT classifier (94.05 F1) are the two strongest reported AD methods reviewed in Phase 2; combining their core ideas (dual-path fusion + dynamic sampling) is the primary model, with Paper 6's retrieval approach as a fallback when no fixed candidate set exists.

Step 6.7 — Algorithm: Concept Extraction (Module 7) — *novel*

This is where the proposed contribution begins (per Phase 3's research gap). Instead of stopping at the full form, the system links the full form to a broader concept.

Input: CNN → Convolutional Neural Network **Output:** Deep Learning

1. Look up (abbreviation, full_form) in the seeded Concept dictionary (Phase 5's Concepts.csv) – direct hit if the term is already known.
2. IF not found:
 - a. Compute an embedding for full_form (SciBERT [CLS] embedding)
 - b. Compare against embeddings of known concept-anchor terms (e.g. "Deep Learning", "Machine Learning", "Natural Language Processing") using cosine similarity
 - c. Assign the concept whose anchor embedding is most similar, above a confidence threshold θ
 - d. IF below threshold: flag for manual/human-in-the-loop labeling

3. Cache the new (full_form -> concept) mapping into Concepts.csv for reuse
RETURN concept

Rationale: No reviewed paper performs this step (Phase 3, Step 3.3) — it is implemented as a hybrid of dictionary lookup (fast, precise, reuses Phase 5’s seed data) and embedding-similarity fallback (generalizes to unseen abbreviations).

Step 6.8 — CCC Mapping Algorithm (Module 8) — *core contribution*

Goal: Instead of storing a flat abbreviation → full_form pair, store a full chain: abbreviation → concept → parent_concept → root_concept.

Example:

```
CNN
↓
Convolutional Neural Network
↓
Deep Learning
↓
Machine Learning
↓
Artificial Intelligence
```

Algorithm:

```
FUNCTION ccc_map(abbreviation, full_form):
    concept = concept_extraction(abbreviation, full_form)      # Step 6.7
    chain = [abbreviation, full_form, concept]

    current = concept
    WHILE current has a parent in the category taxonomy:
        parent = taxonomy_lookup(current)    # e.g. Concepts.csv "Category" field,
                                             # or a curated CS/AI taxonomy (ACM CCS)
        chain.append(parent)
        current = parent

    RETURN chain    # Concept -> Category -> Context (CCC)
```

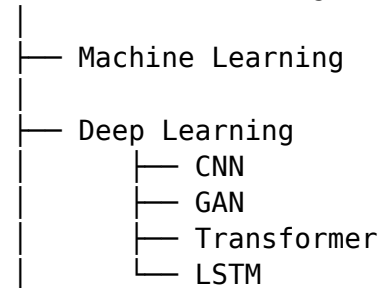
Taxonomy source: initially the Category column already present in Phase 5’s Concepts.csv (e.g. CNN → Deep Learning → Machine Learning); extendable with a standard external taxonomy (e.g. ACM Computing Classification System) once the corpus grows beyond the AI/CS seed domain.

Context component: each CCC chain also stores the originating sentence/section (from Step 6.3) as provenance, so a concept mapping can be traced back to the specific claim in the specific paper that produced it — this is the “Context” in CCC.

Step 6.9 — Hierarchy Generation Algorithm (Module 9)

Input: List of CCC chains (Step 6.8) **Output:** A tree

Artificial Intelligence



Algorithm (parent-child linking + tree construction):

```
FUNCTION build_hierarchy(ccc_chains):
    tree = {} # adjacency map: parent -> set(children)
    FOR chain in ccc_chains:
        FOR i in range(len(chain) - 1, 0, -1): # walk chain root -> leaf
            child, parent = chain[i-1], chain[i]
            tree[parent].add(child)
    root = find_node_with_no_parent(tree) # e.g. "Artificial Intelligence"
    RETURN render_tree(tree, root) # recursive DFS print/serialize
```

Complexity: with n CCC chains of average depth d , building the adjacency map is $O(n \cdot d)$; tree traversal for rendering is $O(V)$ where V is the number of unique concept nodes.

Step 6.10 — Relationship Extraction Algorithm (Module 8b, feeding Module 10)

Beyond the vertical CCC hierarchy, extract lateral relationships between concepts (following Phase 5's Relationships.csv schema).

```
RELATION_PATTERNS = {
    "used_for": [" is used for ", " used to ", " applied to "],
    "belongs_to": ["is a type of", "is a kind of", "belongs to"],
    "part_of": ["is part of", "component of"],
    "extends": ["extends", "builds on", "based on"],
    "improves": ["improves", "outperforms", "outperformed"],
    "depends_on": ["depends on", "relies on", "requires"],
    "similar_to": ["similar to", "comparable to"]
}
```

```
FUNCTION extract_relationships(sentence, concept_A, concept_B):
    FOR relation, patterns in RELATION_PATTERNS:
        IF any pattern found in sentence between concept_A and concept_B:
            RETURN (concept_A, relation, concept_B)
```

```
# fallback: transformer-based relation classifier if no pattern matches
RETURN classify_relation_with_model(sentence, concept_A, concept_B)
```

Relation types: belongs_to, used_for, part_of, extends, improves, depends_on, similar_to — matching Phase 5’s seed Relationships.csv (e.g. hdBERT, outperforms, GAD uses the improves-family pattern).

Step 6.11 — Knowledge Graph Construction (Module 10a)

```
FUNCTION build_knowledge_graph(ccc_chains, relationships):
    G = Graph()    # NetworkX DiGraph
    FOR chain in ccc_chains:
        FOR i in range(len(chain) - 1):
            G.add_edge(chain[i], chain[i+1], relation="belongs_to")
    FOR (source, relation, target) in relationships:
        G.add_edge(source, target, relation=relation)
    RETURN G
```

Example (matches Phase 5 seed data):

```
CNN --belongs_to--> Deep Learning --part_of--> Machine Learning --part_of--> Artificial Intelligence
```

Nodes: every abbreviation, full form, and concept (CNN, Deep Learning, Machine Learning, Artificial Intelligence, ...) **Edges:** belongs_to (from CCC hierarchy), plus used_for, related_to, improves, etc. (from Step 6.10)

Step 6.12 — Visualization Algorithm (Module 10b)

```
FUNCTION visualize(G, mode):
    IF mode == "tree":    RETURN render_tree_layout(G)        # PyVis hierarchical layout
    IF mode == "graph":   RETURN render_force_directed(G)     # Cytoscape / D3.js force lay
    IF mode == "mindmap": RETURN render_radial_layout(G)      # radial tree, root at center
    ADD interactivity: click node -> expand/collapse children,
                        hover edge -> show relation label + source sentence (context)
```

Libraries: NetworkX (graph object + basic layout), PyVis (quick interactive HTML export), Cytoscape.js / D3.js (richer, production interactive front-end) — matching the stack chosen in Phase 4, Step 4.7.

Step 6.13 — Complexity Analysis

Module	Time Complexity	Space Complexity	Notes
PDF Extraction	$O(n)$	$O(n)$	n = number of characters/pages
Text Cleaning	$O(n)$	$O(n)$	n = text length
Tokenization	$O(n)$	$O(n)$	n = text length
Abbreviation Detection (SciBERT tagging)	$O(n \cdot L)$	$O(n)$	n = number of tokens, L = transformer sequence-processing cost per token (effectively $O(n^2)$ within a single attention window of fixed max length, $O(n)$ across windows)
Long Form Extraction	$O(n \cdot w)$	$O(k)$	w = local search window size, k = dictionary size ($O(1)$ amortized with hash lookup)
Disambiguation (hdBERT)	$O(m \cdot L)$	$O(m)$	m = number of ambiguous acronym instances $\times$ candidate count
Concept Extraction	$O(m)$ amortized (dict lookup), $O(m \cdot c)$ worst case	$O(c)$	c = number of concept anchors (embedding comparison fallback)
CCC Mapping	$O(m \cdot d)$	$O(m \cdot d)$	d = average chain depth (typically 3-5)
Hierarchy Generation	$O(V + E)$	$O(V + E)$	V = unique concepts, E = parent-child edges
Relationship Extraction	$O(n \cdot r)$	$O(r)$	r = number of relation patterns checked per sentence pair
Knowledge Graph Construction	$O(V + E)$	$O(V + E)$	standard graph build
Visualization	$O(V + E)$	$O(V + E)$	rendering cost scales with graph size

(These are first-pass estimates per the assignment's own guidance — Step 6.13 notes they should be refined once implementation and profiling are done in a later phase.)

Step 6.14 — Final Algorithm Flow

PDF
↓
Text
↓
Sentences
↓
Tokens
↓
Abbreviations
↓
Full Forms
↓
Disambiguation
↓
Concept Detection
↓
CCC Mapping
↓
Hierarchy
↓
Knowledge Graph
↓
Visualization

Model Selection Summary

Module	Selected Model/Algorithm	Justification (from Phase 2 literature)
Abbreviation Identification	SciBERT + BIO tagging (+ optional FGM adversarial training)	Paper 2 (AT-BERT): 94.12 F1, strongest reviewed AI result
Long Form Extraction	Schwartz & Hearst rule + dictionary lookup	Paper 3 (MadDog): high-precision rule-based approach, no training data needed
Disambiguation	hdBERT-style dual-path fusion + Paper 4's training strategies	Papers 4 & 5: 94.05 and 93.73 F1, the two strongest reviewed AD results

Module	Selected Model/Algorithm	Justification (from Phase 2 literature)
Concept Extraction	Dictionary lookup + embedding-similarity fallback	Novel — no direct precedent; designed to reuse Phase 5 seed data
CCC Mapping	Recursive taxonomy-chain construction	Novel — core proposed contribution (Phase 3)
Hierarchy Generation	Parent-child adjacency + tree traversal	Novel — standard tree-construction algorithm applied to a new data type
Relationship Extraction	Pattern matching + transformer fallback classifier	Novel — pattern set matches Phase 5's Relationships.csv relation types
Knowledge Graph	NetworkX directed graph	Standard, matches Phase 4 tech stack
Visualization	PyVis / Cytoscape.js / D3.js	Standard, matches Phase 4 tech stack; addresses the “no interactive visualization” gap from Phase 3

Deliverables Checklist

- ☒ Overall processing algorithm (Step 6.1, 6.14)
- ☒ Module-wise algorithms (Steps 6.2–6.12)
- ☒ Model selection for each module (Model Selection Summary)
- ☒ CCC mapping algorithm — core contribution (Step 6.8)
- ☒ Hierarchy generation algorithm (Step 6.9)
- ☒ Knowledge graph algorithm (Step 6.11)
- ☒ Visualization strategy (Step 6.12)
- ☒ Complexity analysis (Step 6.13) -e